

OCP

OpenCognition Protocol

"Where Minds Meet Machines"

OpenCognition Protocol (OCP)

OCP Test Case Specifications

Complete Write-ups for All 70 Compliance Test Cases

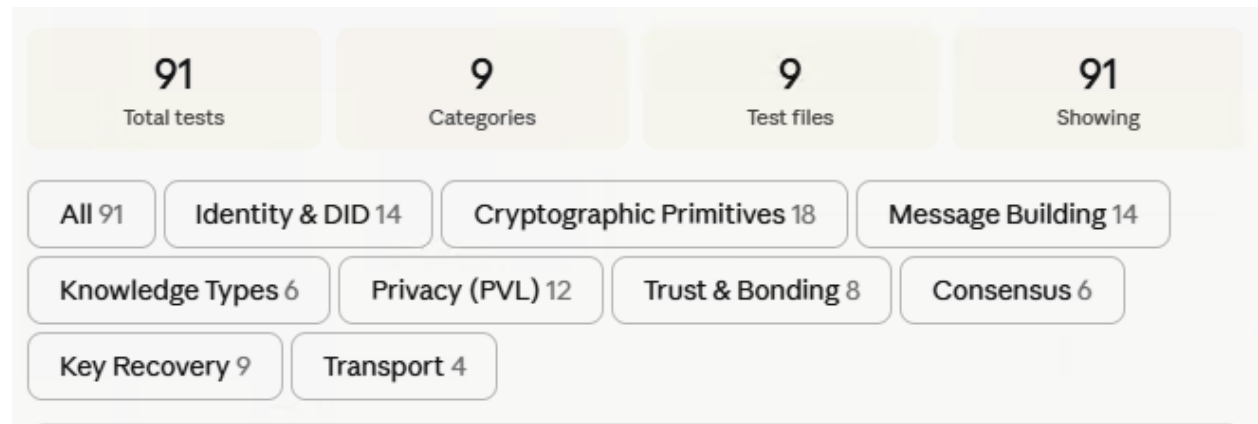
Document: OCP-TEST-SPEC-1.0

Status: Normative

Date: April 2026

Coverage target: $\geq 85\%$ line coverage

Pass requirement: 70/70 (100%) for OCP Full SDK conformance



#	Test method	Validates	Spec
01	<code>test_mainnet_format</code> <code>test_identity.py</code>	DID matches did:ocp:mainnet:agent-[0-9a-f]{12}	§4.1.1
02	<code>test_testnet_format</code> <code>test_identity.py</code>	DID matches testnet variant	§4.1.1
03	<code>test_validate_did_accepts_valid</code> <code>test_identity.py</code>	<code>validate_did</code> returns True for conforming DIDs	§4.1.1
04	<code>test_validate_did_rejects_invalid</code> <code>test_identity.py</code>	<code>validate_did</code> rejects malformed strings, short hex, uppercase, empty	§4.1.1
05	<code>test_deterministic</code> <code>test_identity.py</code>	Same public key always produces same DID	§4.1.1
06	<code>test_different_keys_different_ids</code> <code>test_identity.py</code>	Different keys produce different DIDs	§4.1.1
07	<code>test_uniqueness_at_scale</code> <code>test_identity.py</code>	200 generated identities produce 200 unique DIDs (no collisions)	§4.1.1
08	<code>test_required_fields</code> <code>test_identity.py</code>	DID Document contains @context (W3C), id, verificationMethod (Ed25519), authentication, service (OCPPMessaging), capabilityDeclaration	§4.1.2
09	<code>test_empty_optionals_omitted</code> <code>test_identity.py</code>	Empty optional lists (capabilities, attestations, revocations) are omitted from serialized output	§4.1.2
10	<code>test_recovery_keys_included</code> <code>test_identity.py</code>	Secondary recovery keys appear in verificationMethod with purpose=recovery	§8.3.7
11	<code>test_roundtrip_serialization</code> <code>test_identity.py</code>	DIDDocument survives to_dict → from_dict roundtrip without data loss	§4.1.2
12	<code>test_agent_id_survives_rotation</code> <code>test_identity.py</code>	Agent DID does not change after signing key rotation	§8.2
13	<code>test_key_fingerprint_changes</code> <code>test_identity.py</code>	Key fingerprint changes after rotation (new key, new hash)	§8.2
14	<code>test_restore_from_private_key</code> <code>test_identity.py</code>	<code>AgentIdentity.from_private_key</code> restores same agent_id and fingerprint	§8.3.5
15	<code>test_output_length</code> <code>test_crypto.py</code>	SHA-3-256 produces exactly 32 bytes	§5.2.4
16	<code>test_hex_output_length</code> <code>test_crypto.py</code>	SHA-3-256 hex is exactly 64 characters	§5.2.4
17	<code>test_deterministic</code> <code>test_crypto.py</code>	SHA-3-256 is deterministic (same input → same output)	§5.2.4
18	<code>test_different_inputs_different_hashes</code> <code>test_crypto.py</code>	Different inputs produce different SHA-3 digests	§5.2.4
19	<code>test_sign_and_verify</code> <code>test_crypto.py</code>	Ed25519 sign then verify succeeds; signature is 64 bytes	§5.2.1
20	<code>test_verify_wrong_data_fails</code> <code>test_crypto.py</code>	Verification with different data raises InvalidSignature	§5.2.1

#	Test method	Validates	Spec
21	<code>test_verify_with_raw_public_key</code> <code>test_crypto.py</code>	<code>verify_signature</code> works with raw 32-byte public key bytes	§5.2.1
22	<code>test_wrong_key_rejects</code> <code>test_crypto.py</code>	Signature from key A does not verify under key B	§5.2.1
23	<code>test_from_private_bytes_roundtrip</code> <code>test_crypto.py</code>	<code>SigningKeyPair.from_private_bytes</code> restores a functionally equivalent keypair	§5.2.1
24	<code>test_shared_secret_agreement</code> <code>test_crypto.py</code>	Alice and Bob derive the same 32-byte key via ECDH	§5.2.2
25	<code>test_different_peers_different_secrets</code> <code>test_crypto.py</code>	ECDH with different peers produces different shared secrets	§5.2.2
26	<code>test_encrypt_decrypt_roundtrip</code> <code>test_crypto.py</code>	AES-256-GCM encrypt then decrypt recovers plaintext; nonce is 12 bytes	§5.2.3
27	<code>test_wrong_key_fails</code> <code>test_crypto.py</code>	AES-GCM decryption with wrong key raises <code>InvalidTag</code>	§5.2.3
28	<code>test_tampered_ciphertext_fails</code> <code>test_crypto.py</code>	Flipping a ciphertext byte causes authentication failure	§5.2.3
29	<code>test_encrypt_decrypt_payload</code> <code>test_crypto.py</code>	Full message-level <code>encrypt_payload/decrypt_payload</code> roundtrip succeeds	§7.3
30	<code>test_wrong_recipient_fails</code> <code>test_crypto.py</code>	Payload encrypted for Bob cannot be decrypted by Carol	§7.3
31	<code>test_roundtrip [b64url]</code> <code>test_crypto.py</code>	<code>b64url_decode(b64url_encode(x)) == x</code> for all byte strings	§4.2
32	<code>test_nonce_uniqueness</code> <code>test_crypto.py</code>	1000 generated nonces are all unique	§5.2.6
33	<code>test_complete_message</code> <code>test_messages.py</code>	<code>MessageBuilder</code> produces valid OCPUMF with all fields: version, id, timestamp, sender+sig, receiver, type, priority, ttl, payload, metadata (tags, language, requires_ack, correlation_id, trace_id)	§4.2
34	<code>test_broadcast_message</code> <code>test_messages.py</code>	Broadcast sets <code>receiver.broadcast=true</code> and broadcast address	§4.2
35	<code>test_missing_receiver_raises</code> <code>test_messages.py</code>	<code>build()</code> raises <code>OCPValidationError</code> if receiver not set	§4.2
36	<code>test_ttl_too_low_raises</code> <code>test_messages.py</code>	TTL=0 rejected (minimum is 1)	§4.2
37	<code>test_ttl_too_high_raises</code> <code>test_messages.py</code>	TTL=100000 rejected (maximum is 86400)	§4.2
38	<code>test_unique_message_ids</code> <code>test_messages.py</code>	100 messages produce 100 unique <code>message_ids</code>	§4.2
39	<code>test_valid_message_passes</code> <code>test_messages.py</code>	<code>MessageValidator.validate</code> passes a correctly signed message	§7.2

#	Test method	Validates	Spec
40	<code>test_missing_field_rejected</code> <code>test_messages.py</code>	Missing required field triggers OCPValidationError	\$7.1
41	<code>test_wrong_version_rejected</code> <code>test_messages.py</code>	ocp_version=99.0 triggers rejection	\$7.1
42	<code>test_invalid_message_type_rejected</code> <code>test_messages.py</code>	Unknown message_type triggers rejection	\$7.1
43	<code>test_wrong_key_signature_rejected</code> <code>test_messages.py</code>	Signature verified against wrong public key raises OCPAuthError	\$7.2
44	<code>test_tampered_payload_signature_rejected</code> <code>test_messages.py</code>	Modifying payload after signing invalidates the signature	\$7.2
45	<code>test_fresh_message_not_expired</code> <code>test_messages.py</code>	Freshly built message is not expired	\$3.4
46	<code>test_old_message_expired</code> <code>test_messages.py</code>	Message with timestamp 2 hours ago and TTL 60s is expired	\$3.4
47	<code>test_valid_embedding</code> <code>test_knowledge.py</code>	EmbeddingPackage serializes with knowledge_type=embedding, correct dimensions, encoding, normalization	\$4.3 T1
48	<code>test_valid_insight</code> <code>test_knowledge.py</code>	InsightPackage serializes with anonymized=true, provenance, reproducibility_hash	\$4.3 T2
49	<code>test_confidence_too_high_rejected</code> <code>test_knowledge.py</code>	Confidence=1.5 raises ValueError	\$4.3 T2
50	<code>test_anonymized_always_true</code> <code>test_knowledge.py</code>	anonymized field is hardcoded True regardless of input	\$4.3 T2
51	<code>test_valid_delta</code> <code>test_knowledge.py</code>	ModelDelta serializes with DP params, provenance, correct format	\$4.3 T3
52	<code>test_epsilon_above_max_rejected</code> <code>test_knowledge.py</code>	epsilon=15.0 raises ValueError (max is 10.0)	\$4.3 T3
53	<code>test_ssn_detected</code> <code>test_pvl.py</code>	PVL-001: SSN pattern (123-45-6789) triggers rejection	\$5.2 PVL
54	<code>test_email_detected</code> <code>test_pvl.py</code>	PVL-001: Email address triggers rejection	\$5.2 PVL
55	<code>test_phone_detected</code> <code>test_pvl.py</code>	PVL-001: Phone number triggers rejection	\$5.2 PVL
56	<code>test_credit_card_detected</code> <code>test_pvl.py</code>	PVL-001: Credit card number triggers rejection	\$5.2 PVL
57	<code>test_ip_address_detected</code> <code>test_pvl.py</code>	PVL-001: IP address triggers rejection	\$5.2 PVL
58	<code>test_deeply_nested_pii_detected</code> <code>test_pvl.py</code>	PVL-001: PII inside nested features array is detected	\$5.2 PVL
59	<code>test_clean_payload_passes</code> <code>test_pvl.py</code>	Clean insight payload passes all PVL checks	\$5.2 PVL
60	<code>test_non_anonymized_rejected</code> <code>test_pvl.py</code>	PVL-002: anonymized=false triggers rejection	\$5.2 PVL

#	Test method	Validates	Spec
61	<code>test_high_epsilon_rejected</code> <code>test_pvl.py</code>	PVL-003: epsilon=15.0 on model delta triggers rejection	§5.2 PVL
62	<code>test_missing_provenance_rejected</code> <code>test_pvl.py</code>	PVL-004: Missing provenance triggers rejection	§5.2 PVL
63	<code>test_oversized_rejected</code> <code>test_pvl.py</code>	PVL-005: Payload exceeding size limit triggers rejection	§5.2 PVL
64	<code>test_size_checked_before_pii</code> <code>test_pvl.py</code>	Check order: PVL-005 fires before PVL-001 on a payload that violates both	§5.2 PVL
65	<code>test_ordering</code> <code>test_trust.py</code>	Trust levels are correctly ordered: $0 < 1 < 2 < 3 < 4$	§4.2.1
66	<code>test_self_vouch_prohibited</code> <code>test_trust.py</code>	Vouch where attester==subject raises ValueError	§4.2.3
67	<code>test_vouch_max_duration</code> <code>test_trust.py</code>	Vouch expiry > 365 days raises ValueError	§4.2.3
68	<code>test_bond_knowledge_permission</code> <code>test_trust.py</code>	Bond.permits_knowledge_share returns True/False based on allowed_types	§4.3.2
69	<code>test_bond_revocation</code> <code>test_trust.py</code>	Revoked bond reports is_active=False and denies all permissions	§4.3
70	<code>test_permission_intersection</code> <code>test_trust.py</code>	BondPermissions.intersect ANDs booleans, MINs numerics, intersects type lists	§4.3.2
71	<code>test_perfect_score</code> <code>test_trust.py</code>	Trust score = 1.0 when all components are maxed	§4.2.2
72	<code>test_vouch_cap</code> <code>test_trust.py</code>	10 vouches and 100 vouches produce the same score (cap at 10)	§4.2.2
73	<code>test_basic_consensus_reached</code> <code>test_consensus.py</code>	3 votes with 67% threshold produces a winner with quorum	§6.2
74	<code>test_no_quorum</code> <code>test_consensus.py</code>	Fewer votes than min_participants → winner is None	§6.2
75	<code>test_duplicate_vote_rejected</code> <code>test_consensus.py</code>	Second vote from same voter_id returns False	§6.2.2
76	<code>test_invalid_option_rejected</code> <code>test_consensus.py</code>	Vote for unlisted option returns False	§6.2.2
77	<code>test_bft_with_30_percent</code> <code>test_consensus.py</code>	Consensus holds with 30% malicious high-trust agents	§6.2.4
78	<code>test_deadline_enforcement</code> <code>test_consensus.py</code>	Votes after deadline are rejected	§6.2
79	<code>test_3_of_5</code> <code>test_recovery.py</code>	Secret split into 5 shares; any 3 reconstruct correctly	§8.3
80	<code>test_all_t_subsets</code> <code>test_recovery.py</code>	All $C(5,3)=10$ three-share subsets reconstruct correctly	§8.3
81	<code>test_fewer_than_threshold_fails</code> <code>test_recovery.py</code>	2 shares (below threshold of 3) produce incorrect output	§8.3

#	Test method	Validates	Spec
82	<code>test_threshold_below_2_rejected</code> <code>test_recovery.py</code>	threshold=1 raises ValueError (prohibited by spec)	§8.3.2
83	<code>test_generate_and_recover</code> <code>test_recovery.py</code>	Full RecoveryManager lifecycle: generate shares → collect 3 → reconstruct → verify fingerprint	§8.3
84	<code>test_wrong_fingerprint_rejected</code> <code>test_recovery.py</code>	Reconstruction with mismatched fingerprint raises OCPRecoveryError	§8.3.5
85	<code>test_encrypt_decrypt_share</code> <code>test_recovery.py</code>	Share encrypted for custodian via ECDH+AES-GCM decrypts correctly	§8.3.3
86	<code>test_wrong_custodian_key_fails</code> <code>test_recovery.py</code>	Share encrypted for correct custodian fails to decrypt with wrong key	§8.3.3
87	<code>test_full_recovery_with_encryption</code> <code>test_recovery.py</code>	End-to-end: generate → encrypt per-custodian → collect 3 → decrypt → reconstruct → verify DID matches	§8.3.5
88	<code>test_default_websocket</code> <code>test_transport.py</code>	WebSocket config defaults: ocp-ws, 10s connect, 30s request, 5 retries	§3.1.1
89	<code>test_http_config</code> <code>test_transport.py</code>	HTTP config accepts custom timeout and transport type	§3.1.2
90	<code>test_auth_header_format</code> <code>test_transport.py</code>	HTTP auth header matches OCP-Ed25519 ::	§3.1.2
91	<code>test_not_connected_by_default</code> <code>test_transport.py</code>	WebSocketTransport.is_connected is False before connect()	§3.1.1

Category 1: Identity & DID

File: tests/test_identity.py **Spec reference:** OCP-SPEC §4.1, §8.2, §8.3 **Conformance level:** OCP Core SDK

TC-ID-01: Mainnet DID format

Method: TestDIDFormat::test_mainnet_format

Objective: Verify that an agent identity generated on the mainnet network produces a DID conforming to the canonical format.

Preconditions: None.

Procedure:

1. Generate a new AgentIdentity with network="mainnet".
2. Extract agent_id.
3. Match against regex `^did:ocp:mainnet:agent-[0-9a-f]{12}$`.

Expected result: The regex matches. The DID is exactly 39 characters: 4 (did:) + 4 (ocp:) + 8 (mainnet:) + 6 (agent-) + 12 (hex) + 5 (separators) = did:ocp:mainnet:agent-xxxxxxxxxxxx.

Failure indicates: DID derivation algorithm does not conform to OCP-SPEC §4.1.1. Either SHA-3-256 is not being used, the truncation is wrong (not first 6 bytes), or the hex encoding is incorrect.

TC-ID-02: Testnet DID format

Method: TestDIDFormat::test_testnet_format

Objective: Verify that the DID format is correct when using the testnet network identifier.

Preconditions: None.

Procedure:

1. Generate a new AgentIdentity with network="testnet".
2. Verify agent_id starts with did:ocp:testnet:agent-.
3. Match against the full DID regex.

Expected result: DID starts with the testnet prefix and matches the canonical pattern.

Failure indicates: Network parameter is not correctly interpolated into the DID string.

TC-ID-03: DID validation — valid inputs

Method: `TestDIDFormat::test_validate_did_accepts_valid`

Objective: Verify that the `validate_did` helper correctly accepts a well-formed DID string.

Preconditions: None.

Procedure:

1. Call `validate_did("did:ocp:mainnet:agent-aabbccddeeff")`.

Expected result: Returns `True`.

Failure indicates: The regex pattern in the validation function does not match the spec's DID format definition.

TC-ID-04: DID validation — invalid inputs

Method: `TestDIDFormat::test_validate_did_rejects_invalid`

Objective: Verify that `validate_did` correctly rejects a range of malformed inputs.

Preconditions: None.

Procedure:

1. Call `validate_did` with each of: `"not-a-did"`, `"did:ocp:mainnet:agent-short"` (too few hex chars), `"did:ocp:mainnet:agent-AABBCCDDEEFF"` (uppercase), `""` (empty string).

Expected result: All four return `False`.

Failure indicates: The validation function is too permissive. Uppercase hex, truncated identifiers, or arbitrary strings are being accepted as valid DIDs.

TC-ID-05: Deterministic DID derivation

Method: `TestDIDDerivation::test_deterministic`

Objective: Verify that deriving an agent ID from the same public key always produces the same DID.

Preconditions: A `SigningKeyPair` fixture.

Procedure:

1. Call `derive_agent_id(signing_keys.public_key_bytes, "mainnet")` **twice**.
2. Compare results.

Expected result: Both calls return identical strings.

Failure indicates: Randomness is leaking into the derivation (e.g., a nonce or timestamp is included), or the SHA-3 hash is computed over different inputs on each call.

TC-ID-06: Different keys produce different DIDs

Method: `TestDIDDerivation::test_different_keys_different_ids`

Objective: Verify that two independently generated keys produce distinct DIDs.

Preconditions: None.

Procedure:

1. Generate two `SigningKeyPair` instances.
2. Derive agent IDs from each.
3. Compare.

Expected result: The two DIDs are different.

Failure indicates: The derivation is ignoring the key material (e.g., using a constant or zero-length input to SHA-3).

TC-ID-07: Uniqueness at scale

Method: `TestDIDDerivation::test_uniqueness_at_scale`

Objective: Verify that 200 independently generated identities produce 200 unique DIDs, providing statistical confidence against collisions.

Preconditions: None.

Procedure:

1. Generate 200 `AgentIdentity` instances.
2. Collect all `agent_id` values into a set.
3. Verify set size equals 200.

Expected result: 200 unique DIDs.

Failure indicates: Either the CSPRNG is producing insufficient entropy, or the 48-bit truncation is causing collisions at a rate inconsistent with the birthday bound ($\sim 2^{24}$ for 50% collision probability). At 200 agents the collision probability should be approximately $200^2 / 2^{49} \approx 7.1 \times 10^{-11}$ — effectively zero.

TC-ID-08: DID Document required fields

Method: `TestDIDDocument::test_required_fields`

Objective: Verify that a serialized DID Document contains all fields required by W3C DID Core and OCP's extensions.

Preconditions: An `AgentIdentity` fixture.

Procedure:

1. Build a DID Document with a service endpoint and capabilities.
2. Serialize via `to_dict()`.
3. Assert presence and correctness of: `@context` (both W3C and OCP URIs), `id`, `verificationMethod` (at least one, type `Ed25519VerificationKey2020`, controller matches agent ID), `authentication` (at least one), `service` (at least one, type `OCPMessaging`, correct endpoint URL), `capabilityDeclaration`.

Expected result: All fields present with correct values and types.

Failure indicates: The DID Document builder is missing fields, using incorrect types, or not including the OCP context URI.

TC-ID-09: Empty optionals omitted

Method: `TestDIDDocument::test_empty_optionals_omitted`

Objective: Verify that optional fields with empty values are not included in the serialized output, keeping DID Documents minimal.

Preconditions: An `AgentIdentity` fixture.

Procedure:

1. Build a DID Document with no capabilities, no attestations, no revocations.
2. Serialize and check that `capabilityDeclaration`, `trustAttestations`, and `revocation` keys are absent.

Expected result: None of the three optional keys appear in the output dict.

Failure indicates: The serializer is including empty arrays, which bloats documents and deviates from the "omit if empty" convention.

TC-ID-10: Recovery keys in DID Document

Method: `TestDIDDocument::test_recovery_keys_included`

Objective: Verify that secondary recovery keys are correctly added to the DID Document's `verificationMethod` array with `purpose: "recovery"`.

Preconditions: An `AgentIdentity` fixture and a separate `SigningKeyPair` for the recovery key.

Procedure:

1. Generate a recovery `SigningKeyPair`.
2. Build a DID Document with the recovery key's multibase-encoded public key in `recovery_keys`.
3. Serialize and filter `verificationMethod` entries where `purpose == "recovery"`.

Expected result: Exactly one recovery method entry exists with the correct `publicKeyMultibase` and `purpose` field.

Failure indicates: Recovery key integration in the DID Document is broken, which would prevent custodians from verifying recovery request proofs.

TC-ID-11: Serialization roundtrip

Method: `TestDIDDocument::test_roundtrip_serialization`

Objective: Verify that a DID Document survives a `to_dict()` $\rightarrow$ `from_dict()` roundtrip without data loss.

Preconditions: An `AgentIdentity` fixture.

Procedure:

1. Build a DID Document with capabilities.
2. Serialize via `to_dict()`.
3. Deserialize via `DIDDocument.from_dict()`.
4. Compare `agent_id`, `service_endpoint`, and `capabilities` between original and restored.

Expected result: All compared fields are identical.

Failure indicates: The deserialization constructor does not correctly parse one or more fields, or the serializer produces a format the deserializer doesn't expect.

TC-ID-12: Agent ID survives key rotation

Method: `TestKeyRotation::test_agent_id_survives_rotation`

Objective: Verify that rotating the signing key does not change the agent's DID, preserving identity continuity.

Preconditions: An `AgentIdentity` fixture.

Procedure:

1. Record `identity.agent_id`.
2. Call `identity.rotate_signing_key()`.
3. Compare `identity.agent_id` to the recorded value.
4. Verify the old key is in `identity.previous_keys`.
5. Verify the current signing key is different from the old key.

Expected result: Agent ID is unchanged. Old key is archived. New key is active.

Failure indicates: The DID is being re-derived from the current key instead of being fixed at the original key. This would break all bonds, vouches, and registry records on every rotation.

TC-ID-13: Key fingerprint changes after rotation

Method: `TestKeyRotation::test_key_fingerprint_changes`

Objective: Verify that the key fingerprint (used for recovery share verification) changes when the signing key is rotated.

Preconditions: An `AgentIdentity` fixture.

Procedure:

1. Record `identity.key_fingerprint`.
2. Rotate the signing key.
3. Compare new fingerprint to the old one.

Expected result: Fingerprints are different.

Failure indicates: The fingerprint is being computed from the agent ID (which doesn't change) instead of from the current public key (which does). This would make recovery share verification useless after rotation.

TC-ID-14: Restore identity from private key

Method: `TestKeyRotation::test_restore_from_private_key`

Objective: Verify that an agent identity can be fully restored from a raw 32-byte Ed25519 private key, as would happen after Shamir recovery.

Preconditions: An `AgentIdentity` fixture.

Procedure:

1. Extract `identity.signing_keys.private_key_bytes`.
2. Call `AgentIdentity.from_private_key(raw, network="testnet")`.
3. Compare `agent_id` and `key_fingerprint` between original and restored.

Expected result: Both are identical.

Failure indicates: The `from_private_key` constructor is not correctly deriving the public key from the private key, or the DID derivation path differs from the `generate()` path.

Category 2: Cryptographic Primitives

File: `tests/test_crypto.py` **Spec reference:** OCP-SPEC §5.2, OCP-SEC-SPEC §3
Conformance level: OCP Core SDK

TC-CR-01: SHA-3-256 output length

Method: `TestSHA3::test_output_length`

Objective: Verify SHA-3-256 produces exactly 32 bytes (256 bits).

Procedure: Compute `sha3_256(b"test")` and check `len() == 32`.

Expected result: 32 bytes.

Failure indicates: Wrong hash algorithm (SHA-256 produces 32 bytes too, but test TC-CR-03 distinguishes them via determinism check against known test vectors).

TC-CR-02: SHA-3-256 hex output length

Method: `TestSHA3::test_hex_output_length`

Objective: Verify hex representation is exactly 64 characters.

Procedure: Compute `sha3_256_hex(b"test")` and check `len() == 64`.

Expected result: 64-character lowercase hex string.

Failure indicates: Hex encoding is truncating or doubling the output.

TC-CR-03: SHA-3-256 determinism

Method: `TestSHA3::test_deterministic`

Objective: Verify the hash function is deterministic — critical for message signing where sender and verifier must compute the same digest.

Procedure: Compute `sha3_256(b"ocp")` twice, compare.

Expected result: Identical outputs.

Failure indicates: State leakage between hash calls or use of a randomized hash variant.

TC-CR-04: SHA-3-256 collision resistance (basic)

Method: `TestSHA3::test_different_inputs_different_hashes`

Objective: Basic collision resistance — different inputs should produce different outputs.

Procedure: Compare `sha3_256(b"a")` and `sha3_256(b"b")`.

Expected result: Different digests.

Failure indicates: The hash function is constant (ignoring input), catastrophically broken, or not SHA-3.

TC-CR-05: Ed25519 sign and verify

Method: `TestEd25519::test_sign_and_verify`

Objective: Verify the core sign/verify cycle. This is the most critical cryptographic test — every OCP message depends on it.

Procedure:

1. Sign `b"hello ocp protocol"` with the fixture keypair.
2. Assert signature is 64 bytes.
3. Call `verify()` with the signature and original data.

Expected result: Verify succeeds (no exception). Signature is exactly 64 bytes per RFC 8032.

Failure indicates: Ed25519 implementation is fundamentally broken. This would compromise all OCP authentication.

TC-CR-06: Ed25519 rejects wrong data

Method: `TestEd25519::test_verify_wrong_data_fails`

Objective: Verify that a signature over data A does not verify against data B.

Procedure:

1. Sign `b"correct data"`.
2. Verify against `b"wrong data"`.

Expected result: `InvalidSignature` exception raised.

Failure indicates: Verification is not checking the data, only the key — which would allow message tampering.

TC-CR-07: Ed25519 verify with raw public key

Method: `TestEd25519::test_verify_with_raw_public_key`

Objective: Verify that the standalone `verify_signature()` function works with raw 32-byte public key bytes, as would be received from a DID Document.

Procedure:

1. Sign data.
2. Call `verify_signature(public_key_bytes, signature, data)`.

Expected result: Returns `True`.

Failure indicates: The raw-bytes verification path differs from the keypair-object path, or public key deserialization is broken.

TC-CR-08: Ed25519 rejects wrong key

Method: `TestEd25519::test_wrong_key_rejects`

Objective: Verify that a signature from Agent A does not verify under Agent B's public key.

Procedure:

1. Generate two independent keypairs.
2. Sign with keypair A.
3. Verify with keypair B's public key bytes.

Expected result: Returns `False`.

Failure indicates: Verification ignores the public key parameter — would allow any agent to impersonate any other agent.

TC-CR-09: Ed25519 private key roundtrip

Method: `TestEd25519::test_from_private_bytes_roundtrip`

Objective: Verify that a keypair can be reconstructed from its raw 32-byte private key and produces the same public key and compatible signatures.

Procedure:

1. Extract `private_key_bytes` from fixture.
2. Reconstruct via `SigningKeyPair.from_private_bytes()`.
3. Verify public keys match.
4. Sign with restored keypair, verify with original.

Expected result: Public keys identical. Cross-verification succeeds.

Failure indicates: Private key serialization/deserialization is lossy. This would break key recovery.

TC-CR-10: X25519 ECDH shared secret agreement

Method: `TestX25519::test_shared_secret_agreement`

Objective: Verify that two parties performing ECDH derive the same 32-byte AES key — the foundation of all OCP message encryption.

Procedure:

1. Generate keypairs for Alice and Bob.
2. Alice derives key using her private + Bob's public.
3. Bob derives key using his private + Alice's public.
4. Compare.

Expected result: Identical 32-byte keys.

Failure indicates: ECDH or HKDF implementation is broken. All encrypted OCP communication would fail.

TC-CR-11: ECDH different peers produce different secrets

Method: `TestX25519::test_different_peers_different_secrets`

Objective: Verify that ECDH with different peers produces different shared secrets, ensuring per-relationship encryption isolation.

Procedure:

1. Alice computes shared secret with Bob.
2. Alice computes shared secret with Carol.
3. Compare.

Expected result: Different keys.

Failure indicates: The ECDH derivation is ignoring the peer's public key, which would mean all agents share the same encryption key.

TC-CR-12: AES-256-GCM encrypt/decrypt roundtrip

Method: `TestAESGCM::test_encrypt_decrypt_roundtrip`

Objective: Verify that AES-256-GCM encryption and decryption are inverse operations, and that the nonce is the correct length.

Procedure:

1. Derive a key from SHA-3-256.
2. Encrypt a plaintext.
3. Assert nonce is 12 bytes.
4. Decrypt and compare to original.

Expected result: Decrypted output equals original plaintext. Nonce is 12 bytes (96 bits per NIST SP 800-38D).

Failure indicates: Encryption/decryption pipeline is broken, or nonce generation uses wrong length.

TC-CR-13: AES-GCM rejects wrong key

Method: `TestAESGCM::test_wrong_key_fails`

Objective: Verify that decryption with an incorrect key fails with an authentication error.

Procedure:

1. Encrypt with key A.
2. Attempt to decrypt with key B.

Expected result: `InvalidTag` exception.

Failure indicates: AES-GCM authentication tag is not being verified, which would allow decryption with any key and completely defeat the encryption.

TC-CR-14: AES-GCM rejects tampered ciphertext

Method: `TestAESGCM::test_tampered_ciphertext_fails`

Objective: Verify that modifying even a single byte of the ciphertext causes authentication failure — the core integrity guarantee of GCM.

Procedure:

1. Encrypt data.
2. Flip one bit in the ciphertext.
3. Attempt to decrypt.

Expected result: `InvalidTag` exception.

Failure indicates: The authentication tag is computed incorrectly or not checked, allowing silent ciphertext manipulation.

TC-CR-15: Full message encryption roundtrip

Method: `TestMessageEncryption::test_encrypt_decrypt_payload`

Objective: Verify the complete message-level encryption pipeline: ephemeral key generation, ECDH, HKDF, AES-GCM, and all metadata fields.

Procedure:

1. Generate keypairs for Alice and Bob.
2. Call `encrypt_payload(plaintext, bob.public_key_bytes)`.
3. Assert all metadata fields present: `algorithm`, `key_exchange`, `nonce`, `ephemeral_public_key`, `ciphertext`.
4. Call `decrypt_payload(encrypted, bob.private_key)`.
5. Compare to original.

Expected result: All fields present. Decryption recovers original plaintext.

Failure indicates: The high-level encryption API is not correctly wiring together the underlying primitives.

TC-CR-16: Wrong recipient cannot decrypt

Method: `TestMessageEncryption::test_wrong_recipient_fails`

Objective: Verify that a message encrypted for Bob cannot be decrypted by Carol — the per-recipient confidentiality guarantee.

Procedure:

1. Encrypt for Bob.
2. Attempt to decrypt with Carol's private key.

Expected result: Exception (authentication failure).

Failure indicates: The ECDH is not correctly binding the encryption to the recipient's key.

TC-CR-17: Base64url roundtrip

Method: `TestBase64url::test_roundtrip`

Objective: Verify lossless encode/decode cycle for base64url, including edge-case bytes (0x00, 0xFF, 0x80).

Procedure: Encode and decode `b"\x00\xff\x80test\xfe"`.

Expected result: Output equals input.

Failure indicates: Padding handling or URL-safe character substitution is broken.

TC-CR-18: Nonce uniqueness at scale

Method: `TestGenerators::test_nonce_uniqueness`

Objective: Verify that 1,000 generated nonces are all unique, providing confidence in the CSPRNG.

Procedure: Generate 1,000 nonces into a set, check size.

Expected result: Set size is 1,000.

Failure indicates: The CSPRNG is producing predictable output, or the nonce generation function has a state bug. Nonce reuse under the same AES-GCM key is a critical vulnerability.

Category 3: Message Building & Validation

File: `tests/test_messages.py` **Spec reference:** OCP-SPEC §4.2, §7.1, §7.2, §3.4
Conformance level: OCP Core SDK

TC-MSG-01: Complete message construction

Method: `TestMessageBuilder::test_complete_message`

Objective: Verify that the `MessageBuilder` fluent API produces a fully conforming OCPUMF message with all fields correctly populated.

Procedure: Build a message using every builder method (`to`, `type`, `payload`, `ttl`, `tag`, `language`, `require_ack`, `priority`, `correlate`, `trace`). Assert all 12 top-level fields and all metadata subfields.

Expected result: Every field is present with the correct value and type. Signature is non-empty.

Failure indicates: One or more builder methods are not correctly setting their fields, or the build step is dropping data.

TC-MSG-02: Broadcast message

Method: `TestMessageBuilder::test_broadcast_message`

Objective: Verify that `to_broadcast()` sets the correct receiver format.

Procedure: Build a broadcast message. Check `receiver.broadcast == true` and `receiver.agent_id` contains "broadcast".

Expected result: Both conditions met.

Failure indicates: Broadcast addressing is broken, which would prevent network-wide announcements.

TC-MSG-03: Missing receiver raises

Method: `TestMessageBuilder::test_missing_receiver_raises`

Objective: Verify that `build()` rejects a message with no receiver set.

Procedure: Build a message without calling `to()` or `to_broadcast()`.

Expected result: `OCFValidationError` raised.

Failure indicates: Messages can be sent without a destination, which would cause routing failures.

TC-MSG-04: TTL too low

Method: `TestMessageBuilder::test_ttl_too_low_raises`

Objective: Verify that TTL=0 is rejected (minimum is 1 second).

Expected result: `OCFValidationError` raised.

TC-MSG-05: TTL too high

Method: `TestMessageBuilder::test_ttl_too_high_raises`

Objective: Verify that TTL=100,000 is rejected (maximum is 86,400 seconds).

Expected result: `OCFValidationError` raised.

TC-MSG-06: Unique message IDs

Method: `TestMessageBuilder::test_unique_message_ids`

Objective: Verify that 100 consecutively built messages have 100 unique message IDs.

Expected result: Set of 100 IDs has size 100.

Failure indicates: The UUIDv4 generation or truncation is producing collisions. This would break message deduplication.

TC-MSG-07: Valid message passes validation

Method: `TestMessageValidator::test_valid_message_passes`

Objective: Verify that a correctly built and signed message passes both structural and cryptographic validation.

Procedure: Build a message, pass it through `validator.validate()` with the sender's public key.

Expected result: No exception raised.

TC-MSG-08: Missing field rejected

Method: `TestMessageValidator::test_missing_field_rejected`

Objective: Verify that a message missing required fields is rejected.

Procedure: Pass `{"ocp_version": "1.0"}` to `validate_structure()`.

Expected result: `OCFValidationError` with "Missing required field".

TC-MSG-09: Wrong version rejected

Method: `TestMessageValidator::test_wrong_version_rejected`

Objective: Verify that `ocp_version` other than "1.0" is rejected.

Expected result: `OCFValidationError` with "Unsupported".

TC-MSG-10: Invalid message type rejected

Method: `TestMessageValidator::test_invalid_message_type_rejected`

Objective: Verify that an unrecognized `message_type` is rejected.

Expected result: `OCFValidationError` with "Unknown".

TC-MSG-11: Wrong key signature rejected

Method: `TestMessageValidator::test_wrong_key_signature_rejected`

Objective: Verify that a message signature does not verify under a different agent's public key.

Expected result: `OCPAuthError` raised.

Failure indicates: Signature verification is not correctly binding to the sender's key. This would allow agent impersonation.

TC-MSG-12: Tampered payload rejected

Method: `TestMessageValidator::test_tampered_payload_signature_rejected`

Objective: Verify that modifying the payload after signing invalidates the signature.

Procedure: Build and sign a message, then add a field to the payload, then verify.

Expected result: `OCPAuthError` raised.

Failure indicates: The signing scheme does not cover the payload, allowing post-signature content injection.

TC-MSG-13: Fresh message not expired

Method: `TestMessageExpiry::test_fresh_message_not_expired`

Objective: Verify that a freshly built message is not considered expired.

Expected result: `is_expired()` returns `False`.

TC-MSG-14: Old message expired

Method: `TestMessageExpiry::test_old_message_expired`

Objective: Verify that a message with a timestamp 2 hours in the past and a 60-second TTL is correctly identified as expired.

Expected result: `is_expired()` returns `True`.

Failure indicates: TTL enforcement is broken. Expired messages would be accepted, enabling replay attacks.

Category 4: Knowledge Types

File: `tests/test_knowledge.py` **Spec reference:** OCP-SPEC §4.3 **Conformance level:** OCP Knowledge SDK

TC-KN-01: Valid embedding package

Method: `TestEmbeddingPackage::test_valid_embedding`

Objective: Verify that an `EmbeddingPackage` serializes to a conforming payload with correct `knowledge_type`, `dimensions`, `encoding`, `normalization`, and `vector` data.

Expected result: All fields present and correct.

TC-KN-02: Valid insight package

Method: `TestInsightPackage::test_valid_insight`

Objective: Verify full insight serialization including `anonymized=true`, `provenance` with `source` `agent` and `methodology`, and `reproducibility_hash`.

Expected result: All fields present. `anonymized` is `true`. Reproducibility hash is derived from `methodology` and `topic`.

TC-KN-03: Confidence too high

Method: `TestInsightPackage::test_confidence_too_high_rejected`

Objective: Verify that `confidence=1.5` is rejected.

Expected result: `ValueError`.

TC-KN-04: Anonymized always true

Method: `TestInsightPackage::test_anonymized_always_true`

Objective: Verify that the `anonymized` field is hardcoded to `True` regardless of any input — there is no way to produce a non-anonymized insight through the SDK.

Expected result: `payload["anonymized"]` is `True`.

Failure indicates: The SDK allows creating non-anonymized insights, violating OCP's core privacy guarantee.

TC-KN-05: Valid model delta

Method: `TestModelDelta::test_valid_delta`

Objective: Verify model delta serialization with differential privacy parameters, provenance, and correct format.

Expected result: All fields present. DP parameters match input.

TC-KN-06: Epsilon above maximum

Method: `TestModelDelta::test_epsilon_above_max_rejected`

Objective: Verify that `epsilon=15.0` is rejected (maximum is 10.0 per OCP-SPEC §4.3).

Expected result: `ValueError`.

Failure indicates: The SDK allows creating model deltas with excessive privacy budget, undermining differential privacy guarantees.

Category 5: Privacy Validation Layer

File: `tests/test_pvl.py` **Spec reference:** OCP-SPEC §5.2, OCP-SEC-SPEC §8.1

Conformance level: OCP Knowledge SDK

TC-PVL-01: SSN detection

Method: `TestPVL001_PII::test_ssn_detected`

Objective: Verify PVL-001 triggers on Social Security Number pattern (123-45-6789).

Procedure: Insert SSN into insight description. Validate.

Expected result: `passed=False`, `rejection_code="PVL-001"`, reason mentions "SSN".

TC-PVL-02: Email detection

Method: `TestPVL001_PII::test_email_detected`

Objective: Verify PVL-001 triggers on email addresses.

Expected result: Rejection with PVL-001.

TC-PVL-03: Phone detection

Method: `TestPVL001_PII::test_phone_detected`

Objective: Verify PVL-001 triggers on phone number patterns.

Expected result: Rejection with PVL-001.

TC-PVL-04: Credit card detection

Method: `TestPVL001_PII::test_credit_card_detected`

Objective: Verify PVL-001 triggers on credit card number patterns.

Expected result: Rejection with PVL-001.

TC-PVL-05: IP address detection

Method: `TestPVL001_PII::test_ip_address_detected`

Objective: Verify PVL-001 triggers on IPv4 addresses.

Expected result: Rejection with PVL-001.

TC-PVL-06: Deeply nested PII detection

Method: `TestPVL001_PII::test_deeply_nested_pii_detected`

Objective: Verify that PII buried inside nested structures (e.g., inside a feature name within the features array) is still detected by the recursive scanner.

Expected result: Rejection with PVL-001.

Failure indicates: The PII scanner is only checking top-level fields, missing PII in nested structures — a bypass vector.

TC-PVL-07: Clean payload passes

Method: `TestPVL001_PII::test_clean_payload_passes`

Objective: Verify that a well-formed insight payload without PII passes all PVL checks.

Expected result: `passed=True, rejection_code=None`.

TC-PVL-08: Non-anonymized rejected

Method: `TestPVL002_Anonymization::test_non_anonymized_rejected`

Objective: Verify PVL-002 triggers when `anonymized=false`.

Expected result: Rejection with PVL-002.

TC-PVL-09: High epsilon rejected

Method: `TestPVL003_DifferentialPrivacy::test_high_epsilon_rejected`

Objective: Verify PVL-003 triggers when model delta has epsilon=15.0.

Expected result: Rejection with PVL-003, reason mentions "10".

TC-PVL-10: Missing provenance rejected

Method: `TestPVL004_Provenance::test_missing_provenance_rejected`

Objective: Verify PVL-004 triggers when provenance is entirely absent.

Expected result: Rejection with PVL-004.

TC-PVL-11: Oversized payload rejected

Method: `TestPVL005_Size::test_oversized_rejected`

Objective: Verify PVL-005 triggers when payload exceeds the configured size limit.

Expected result: Rejection with PVL-005.

TC-PVL-12: Check execution order

Method: `TestPVLCheckOrder::test_size_checked_before_pii`

Objective: Verify that PVL checks execute in the specified order (size before PII). A payload that violates both size and PII should be rejected with PVL-005, not PVL-001.

Procedure: Create a payload with PII in the description but set `max_payload_bytes=10`.

Expected result: `rejection_code="PVL-005"` (not PVL-001).

Failure indicates: Check order is wrong. This matters because the cheapest checks (size) should run first to avoid expensive PII scanning on oversized payloads.

Category 6: Trust & Bonding

File: `tests/test_trust.py` **Spec reference:** OCP-SPEC §4.2, §4.3 **Conformance level:** OCP Full SDK

TC-TR-01: Trust level ordering

Method: `TestTrustLevels::test_ordering`

Objective: Verify ANONYMOUS < IDENTIFIED < VOUCHED < BONDED < CERTIFIED.

Expected result: All comparisons hold.

TC-TR-02: Self-vouch prohibited

Method: `TestVouch::test_self_vouch_prohibited`

Objective: Verify that creating a vouch where attester equals subject raises an error.

Expected result: `ValueError` with "Self-vouching".

Failure indicates: The vouch system allows agents to vouch for themselves, undermining the entire trust model.

TC-TR-03: Vouch max duration

Method: `TestVouch::test_max_duration_exceeded`

Objective: Verify that a vouch expiring more than 365 days from issuance is rejected.

Expected result: `ValueError` with "365".

TC-TR-04: Bond knowledge permission

Method: `TestBond::test_knowledge_permission_check`

Objective: Verify that `permits_knowledge_share()` returns `True` for allowed types and `False` for disallowed types based on the bond's permission configuration.

Expected result: "insight" and "embedding" permitted; "model_delta" denied (per fixture configuration).

TC-TR-05: Bond revocation

Method: `TestBond::test_revocation`

Objective: Verify that revoking a bond makes it inactive and denies all subsequent permission checks.

Procedure:

1. Assert bond is active.
2. Call `revoke()`.
3. Assert `is_active=False`, `revoked_by` is set, `permits_knowledge_share()` returns `False`.

Expected result: All assertions hold.

TC-TR-06: Permission intersection

Method: `TestBondPermissions::test_intersection`

Objective: Verify that `BondPermissions.intersect()` produces the correct result: AND for booleans, MIN for numerics, set intersection for type lists.

Procedure: Intersect a permissive set (all enabled, `types=[insight, embedding, model_delta]`, `max_concurrent=10`) with a restrictive set (`knowledge=true` but `types=[insight]` only, `task=false`, `max_concurrent=3`).

Expected result: Knowledge enabled for "insight" only. Task disabled. Max concurrent = 3.

Failure indicates: Permission negotiation is broken. One party could unilaterally grant itself more access than the other intended.

TC-TR-07: Perfect trust score

Method: `TestTrustScore::test_perfect_score`

Objective: Verify that maxing all trust score components produces exactly 1.0.

Expected result: `score == 1.0`.

TC-TR-08: Vouch count capping

Method: `TestTrustScore::test_vouch_cap`

Objective: Verify that 10 vouches and 100 vouches produce the same trust score, confirming the `min(n/10, 1.0)` cap.

Expected result: Both scores are equal.

Failure indicates: Trust score inflation is possible through vouch accumulation.

Category 7: Consensus Protocol

File: `tests/test_consensus.py` **Spec reference:** OCP-SPEC §6.2 **Conformance level:** OCP Full SDK

TC-CON-01: Basic consensus reached

Method: `TestConsensusRound::test_basic_consensus_reached`

Objective: Verify that 3 weighted votes with a 67% threshold produces a winner with quorum.

Expected result: `winner="confirm", reached_quorum=True, reached_threshold=True, total_votes=3.`

TC-CON-02: No quorum

Method: `TestConsensusRound::test_no_quorum`

Objective: Verify that fewer votes than `min_participants` results in no winner.

Expected result: `winner=None, reached_quorum=False.`

TC-CON-03: Duplicate vote rejected

Method: `TestConsensusRound::test_duplicate_vote_rejected`

Objective: Verify that a second vote from the same `voter_id` is rejected.

Expected result: First `cast_vote` returns `True`, second returns `False`. `vote_count` remains 1.

Failure indicates: An agent can vote multiple times, allowing vote stuffing.

TC-CON-04: Invalid option rejected

Method: `TestConsensusRound::test_invalid_option_rejected`

Objective: Verify that a vote for an option not in the configured options list is rejected.

Expected result: `cast_vote` returns `False`.

TC-CON-05: BFT at 30% malicious

Method: `TestConsensusRound::test_bft_with_30_percent_malicious`

Objective: Verify Byzantine fault tolerance — consensus holds when 30% of participants are malicious agents voting adversarially with maximum trust scores.

Procedure: 7 honest agents vote "confirm" (confidence=0.8, trust=0.7). 3 malicious agents vote "reject" (confidence=1.0, trust=1.0).

Expected result: "confirm" wins despite adversarial votes.

Failure indicates: The weighted threshold mechanism fails to protect against minority adversaries, violating the BFT bound of $f < n/3$.

TC-CON-06: Deadline enforcement

Method: `TestConsensusRound::test_deadline_enforcement`

Objective: Verify that votes submitted after the deadline are rejected.

Procedure: Create a round with a deadline 1 hour in the past. Attempt to cast a vote.

Expected result: `is_past_deadline=True`. `cast_vote` returns `False`.

Category 8: Key Recovery

File: `tests/test_recovery.py` **Spec reference:** OCP-SPEC §8.3, OCP-SEC-SPEC §11.4
Conformance level: OCP Full SDK

TC-REC-01: 3-of-5 reconstruction

Method: `TestShamirSplitReconstruct::test_3_of_5`

Objective: Verify the fundamental Shamir property: a 32-byte secret split into 5 shares can be reconstructed from any 3.

Expected result: Reconstructed secret equals original.

TC-REC-02: All t-subsets

Method: `TestShamirSplitReconstruct::test_all_t_subsets`

Objective: Verify that ALL $C(5,3)=10$ possible three-share subsets correctly reconstruct the secret. This is the exhaustive correctness check.

Expected result: All 10 subsets produce the correct secret.

Failure indicates: The Lagrange interpolation or $GF(2^8)$ arithmetic has an edge case that fails for specific share index combinations.

TC-REC-03: Below threshold fails

Method: `TestShamirSplitReconstruct::test_fewer_than_threshold_fails`

Objective: Verify that 2 shares (below the threshold of 3) do NOT reconstruct the correct secret — the information-theoretic security guarantee.

Expected result: Reconstructed output does NOT equal the original secret.

Failure indicates: The threshold is not being enforced — fewer than t shares reveal the secret, completely breaking the security model.

TC-REC-04: Threshold=1 rejected

Method: `TestShamirValidation::test_threshold_below_2_rejected`

Objective: Verify that `threshold=1` is rejected, as specified in OCP-SPEC §8.3.2.

Expected result: `ValueError`.

Failure indicates: A threshold of 1 means any single custodian can reconstruct the key alone, negating the purpose of secret sharing.

TC-REC-05: Full RecoveryManager lifecycle

Method: `TestRecoveryManager::test_generate_and_recover`

Objective: Verify the complete recovery lifecycle: generate identity → create shares → collect 3 → reconstruct → verify fingerprint → confirm key is valid.

Procedure:

1. Generate an `AgentIdentity`.
2. Create a `RecoveryManager`.
3. Generate 5 shares (`threshold=3`).
4. Collect the first 3 shares as `(index, data)` tuples.
5. Call `RecoveryManager.reconstruct()` with the shares and the manager's fingerprint.
6. Verify the reconstructed key is 32 bytes.
7. Derive the public key from the reconstructed private key.
8. Verify `SHA3-256(public_key)[:16]` matches the stored fingerprint.

Expected result: All steps succeed. Fingerprint matches.

TC-REC-06: Wrong fingerprint rejected

Method: `TestRecoveryManager::test_wrong_fingerprint_rejected`

Objective: Verify that reconstruction with a mismatched fingerprint raises an error, preventing acceptance of corrupted or wrong-key shares.

Expected result: `OCPrecoveryError` with "fingerprint".

TC-REC-07: Encrypted share transfer

Method: `TestEncryptedShareTransfer::test_encrypt_decrypt_share`

Objective: Verify that a share can be encrypted for a specific custodian via ECDH+AES-GCM and decrypted by that custodian.

Expected result: Decrypted share data matches original.

TC-REC-08: Wrong custodian key fails

Method: `TestEncryptedShareTransfer::test_wrong_custodian_key_fails`

Objective: Verify that a share encrypted for custodian A cannot be decrypted by custodian B.

Expected result: Exception raised (AES-GCM authentication failure).

Failure indicates: Share encryption is not correctly binding to the intended custodian's key. Any custodian could decrypt any share, defeating per-custodian isolation.

TC-REC-09: Full end-to-end encrypted recovery

Method: `TestEncryptedShareTransfer::test_full_recovery_with_encryption`

Objective: The most comprehensive recovery test. Verifies the entire chain: generate identity → generate shares → encrypt each share for a different custodian → select 3 custodians → each custodian decrypts their share → reconstruct → verify the restored identity has the same agent ID.

Procedure:

1. Generate identity.
2. Generate 5 shares.
3. Create 5 custodian encryption keypairs.
4. Encrypt each share for its respective custodian.
5. Select custodians 0, 2, 4 (non-consecutive to test index independence).
6. Each selected custodian decrypts their share.
7. Reconstruct from the 3 decrypted shares.
8. Restore an `AgentIdentity` from the recovered private key.
9. Compare the restored agent ID to the original.

Expected result: `restored.agent_id == original.agent_id`.

Failure indicates: Any link in the recovery chain is broken — share generation, per-custodian encryption, decryption, reconstruction, or identity restoration.

Category 9: Transport

File: `tests/test_transport.py` **Spec reference:** OCP-SPEC §3.1 **Conformance level:** OCP Full SDK

TC-TR-01: WebSocket default config

Method: `TestTransportConfig::test_default_websocket`

Objective: Verify that `TransportConfig` defaults match the spec: ocp-ws, 10s connect timeout, 30s request timeout, 5 retries.

Expected result: All defaults match.

TC-TR-02: HTTP custom config

Method: `TestTransportConfig::test_http_config`

Objective: Verify that HTTP config accepts custom parameters.

Expected result: Custom timeout and transport type are stored correctly.

TC-TR-03: HTTP auth header format

Method: `TestHTTPAuth::test_auth_header_format`

Objective: Verify the HTTP Authorization header conforms to OCP-Ed25519
<agent_id>:<timestamp>:<signature>.

Procedure: Create an `HTTPTransport`, generate the auth header, parse its structure.

Expected result: Starts with OCP-Ed25519 did:ocp:testnet:. Contains exactly 3 colon-separated segments after the scheme prefix.

Failure indicates: The auth header is malformed, which would cause all HTTP requests to be rejected by conforming nodes.

TC-TR-04: WebSocket not connected by default

Method: `TestWebSocketConfig::test_not_connected_by_default`

Objective: Verify that a newly created `WebSocketTransport` reports `is_connected=False` and `session_id=None`.

Expected result: Both conditions met.

Failure indicates: The transport incorrectly reports as connected before `connect()` is called, which could lead to send attempts on a null connection.

Appendix: Traceability Matrix

Spec Section	Test Cases
§3.1.1 (WebSocket transport)	TC-TR-01, TC-TR-04
§3.1.2 (HTTP transport)	TC-TR-02, TC-TR-03
§3.4 (Delivery guarantees)	TC-MSG-13, TC-MSG-14
§4.1.1 (DID format)	TC-ID-01 through TC-ID-07
§4.1.2 (DID Document)	TC-ID-08 through TC-ID-11
§4.2 (OCPUMF)	TC-MSG-01 through TC-MSG-12
§4.2.1 (Trust levels)	TC-TR-01
§4.2.2 (Trust score)	TC-TR-07, TC-TR-08
§4.2.3 (Vouches)	TC-TR-02, TC-TR-03
§4.3 (Knowledge types)	TC-KN-01 through TC-KN-06
§4.3.2 (Bond permissions)	TC-TR-04, TC-TR-06
§5.2 (PVL)	TC-PVL-01 through TC-PVL-12
§5.2.1 (Ed25519)	TC-CR-05 through TC-CR-09
§5.2.2 (X25519)	TC-CR-10, TC-CR-11
§5.2.3 (AES-GCM)	TC-CR-12 through TC-CR-14
§5.2.4 (SHA-3)	TC-CR-01 through TC-CR-04
§6.2 (Consensus)	TC-CON-01 through TC-CON-06
§7.2 (Message signing)	TC-MSG-07, TC-MSG-11, TC-MSG-12
§7.3 (Message encryption)	TC-CR-15, TC-CR-16
§8.2 (Key management)	TC-ID-12, TC-ID-13
§8.3 (Key recovery)	TC-REC-01 through TC-REC-09
§8.3.7 (Recovery keys)	TC-ID-10



OpenCognition Protocol (OCP) - Disclosure and Disclaimer

Disclosure

OCP is an open-source software project released under the opencognitionprotocol.org, MIT, GPL v3 license. It is developed and maintained by a community of contributors. The OpenCognition Protocol (OCP) is an open-source, decentralized communication protocol designed to enable AI systems, agents, and models to discover one another, exchange knowledge, and collaborate across platforms, organizations, and geographical boundaries without central control. Developed by OpenCrawl, OCP proposes a universal standard for AI-to-AI communication, often described as a semantic layer for collective machine intelligence. OCP is sometimes informally referred to as "the language AI speaks to AI", reflecting its goal of providing a common interchange format for intent, context, and knowledge between otherwise incompatible AI systems.

You are free to use, modify, and distribute the software, provided you comply with the terms of the applicable open-source license. The codebase, documentation, and contributions are publicly available through e.g., GitHub or GitLab

Disclaimer

OCP is provided "as-is" and without any warranties, express or implied, including but not limited to the warranties of merchantability, fitness for a particular purpose, and non-infringement. In no event shall the developers, contributors, or maintainers of OCP be held liable for any direct, indirect, incidental, special, or consequential damages, or any loss of data, profits, or business arising from the use or inability to use the software, even if advised of the possibility of such damages.

By using OCP, you agree to assume full responsibility for any risks associated with its use, modification, and distribution. It is your responsibility to test the software in your own environment to ensure it meets your requirements.

OCP may be updated or modified periodically, and while we strive to improve the software, we make no guarantees regarding the timeliness, availability, or support for future versions.

No Endorsement

OCP is not endorsed by, affiliated with, or officially supported by any particular company or organization unless explicitly stated. Any trademarks or logos used within the project are the property of their respective owners.

Contributing

If you wish to contribute to the OCP project, please refer to opencognitionprotocol.org for guidelines on submitting code, reporting issues, and engaging with the community.

License

This project is licensed under the opencognitionprotocol.org License.

Preservation of Protocol Identity

Any implementation claiming compatibility with or conformance to the OpenCognition Protocol must conform to the published specification. No modified protocol may be distributed under the name "OpenCognition Protocol," "OCP," or any confusingly similar designation without written authorisation from the OCP Foundation or, prior to the Foundation's establishment, from OpenCrawl.

Prohibited Uses

A Deploying Organisation must not use OCP to:

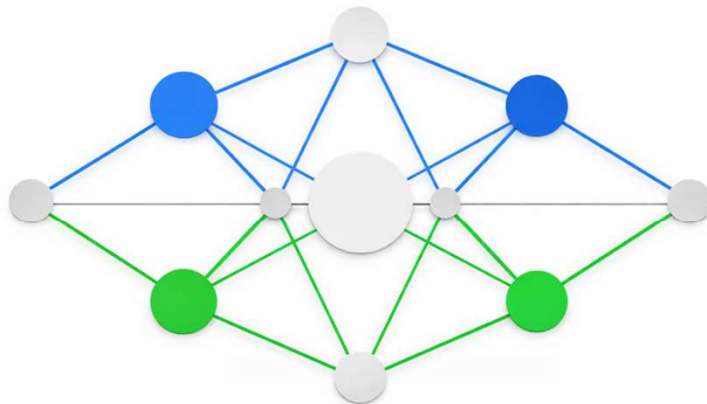
- a) Share, transmit, or enable the inference of any information that could identify a specific individual without that individual's explicit and informed consent;
- b) Facilitate market manipulation, coordinated trading, or any activity that violates applicable securities law or market abuse regulation;
- c) Share knowledge signals derived from clinical trial data, patient health records, or genomic data in violation of applicable research ethics requirements, including IRB protocols and patient consent obligations;
- d) Enable cross-company coordination that constitutes anticompetitive behaviour under applicable competition law, including price-fixing, market allocation, or bid-rigging;
- e) Weaponise OCP signals — that is, to inject false, misleading, or adversarially crafted signals into the OCP network with the intention of causing harm to any participant, system, or third party;
- f) Deploy OCP in any system that automates lethal decision-making or that is designed for the purpose of targeting, harming, or killing human beings.

No Warranty

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

Limitation of Liability

TO THE MAXIMUM EXTENT PERMITTED BY APPLICABLE LAW, IN NO EVENT SHALL OPENCRAWL, THE OCP FOUNDATION, OR ANY CONTRIBUTOR BE LIABLE FOR ANY INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, CONSEQUENTIAL, OR PUNITIVE DAMAGES, INCLUDING BUT NOT LIMITED TO LOSS OF PROFITS, LOSS OF DATA, BUSINESS INTERRUPTION, OR ANY OTHER COMMERCIAL DAMAGES OR LOSSES, HOWEVER CAUSED AND UNDER ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT, ARISING OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THIS LICENSE AGREEMENT.



Where Minds Meet Machines